

└── README.md

架構內容

backend/routing/

Mission

此資料夾是整個專案核心，只負責圖論與交通演算法，必須完整保留 Notebook 中既有邏輯，不可重寫演算法，只能重構為函式。

graph.py

Mission

建立並管理台北市多模式交通 Graph，包含 OSMnx、NetworkX、捷運、公車、YouBike、台鐵、KDTree 等資料載入與 Graph 建立，只負責初始化，不得進行路徑搜尋。

routing.py

Mission

根據 Gemini 回傳的 weights、walking_speed_multiplier 與 banned_vehicles 修改 Graph 的 adjusted_time，呼叫 NetworkX 找出前三條最佳路線，保留 Notebook 原有搜尋方式，不可改變演算法。

walking_speed.py

Mission

保留 Notebook 原有步速公式，依據性別、年齡、身高、體重、BMI 與 Gemini 提供的 `walking_speed_multiplier` 計算實際步速，不得重新設計公式。

`waiting_time.py`

Mission

保留三軌等待時間邏輯：捷運固定等待 2 分鐘、公車固定等待 8 分鐘、台鐵透過 TDX API 查詢下一班列車等待時間，不得修改既有等待時間規則。

`fare.py`

Mission

路徑確定後，依 Notebook 中既有票價公式計算各交通工具票價，完整保留捷運起跳價、火車基本票價、公車段次、敬老與兒童優惠等邏輯，不得簡化或改寫。

`backend/api/`

Mission

所有外部資料來源皆集中於此，不包含任何商業邏輯，只負責向外部 API 取得資料並回傳 Python 物件。

`weather.py`

Mission

呼叫中央氣象署 API 與環境部 API，取得台北市目前天氣、氣溫、降雨、AQI 與警報資訊，整理為標準格式提供其他模組使用，不參與任何決策。

tdx.py

Mission

呼叫 TDX API，取得台鐵、公車或其他公共運輸即時資訊，封裝所有 OAuth、Token 與 API 呼叫細節，不讓其他模組直接存取 TDX。

backend/models/

Mission

建立所有 Pydantic Model，統一管理 API Request、Response 與資料格式，避免各模組自行建立 Dictionary，提高型別安全與維護性。

backend/utils/

Mission

放置專案共用工具，例如 Logger、Geo Helper、時間轉換、距離計算、JSON Helper 等，不可放置任何業務邏輯。

backend/data/

Mission

儲存所有靜態資料，例如 GeoJSON、CSV、Graph Cache、KDTree Cache、站點資料等，不得放置任何 Python 程式。

frontend/

Mission

建立極簡且具現代感的單頁網站，不干擾使用者，只蒐集起點、終點、生理資料、交通工具選擇與自由描述，按下推薦後透過 AJAX 呼叫 FastAPI，並以

Leaflet 呈現前三條路線、真實耗時、票價與 **Gemini** 的同理心 reasoning。使用者可持續輸入新的需求，重新呼叫 **API** 並即時更新結果。

requirements.txt

Mission

僅列出專案實際使用的 **Python** 套件與版本，不安裝未使用的依賴，確保專案可於新環境一鍵安裝。

README.md

Mission

撰寫完整專案說明，包含系統架構、安裝流程、環境變數需求、執行方式、**API** 說明與專案特色，讓新開發者可快速理解並啟動專案。